



Spatial Database Systems

Assignment 2

This assignment is worth 45% of your semester mark.

Assignment brief:

In this assignment, you will extend the work that you have already done in the tutorial sessions based on the railway database.

There are three parts all of which must be completed using postgresSQL. You MAY NOT use the POSTGIS extension for Part A or B. You MUST use the POSTGIS extension for Part C.

GETTING STARTED

Start pgAdmin and create a new database as you have done in the practical sessions. Name it assignment2 – all in lower case. On the Blackboard, where you found this document, I've put a backup of the database you will need for this assignment. Please restore the backup to your new database.

PART A: UPDATE DATABASE (11 marks)

There are two tasks to do in Part A.

Task 1 (11 marks)

Currently, the database contains only one of Perth's train routes: Perth to Fremantle. You need to update the database to include the southern line (running to Mandurah).

This requires 10 new stations shown in Table 1 below. Table 1 also shows station master's name (e.g., Cameron Cooper) and their identification in brackets (e.g., 64MT943).

It is quite possible that you might have to do other things (e.g., alter tables) to get this to work properly. Use your initiative and do them, reporting what you did at each question, but do not add any extra data that you don't know about.

Table 1 - Stations and associated Station Master's (ID's in brackets) that you need to enter into the database.

Station	Station Master
Esplanade	Cameron Cooper (64MT943)
Canning Bridge	Clair Davey (85JW156)
Bull Creek	Diego Fernando (first name) Hernandez Villamil (last name) (79VH346)
Murdoch	Wai Kit (first name) Lau (last name) (74AP984)
Cockburn Central	Jordan Maddock (89QF747)
Kwinana	Tika Ram (first name) Mahatso (last name) (25SW748)
Wellard	Dunusinhege Taniya (first name) Nawagamuwa (last name) (25SW219)
Rockingham	Jennifer Ann (first name) Potucek (last name) (59DQ592)
Warnbro	Thinley Paday (26AB623)
Mandurah	Amanda Tevlone (19OC305)

1. Create a table called **Master** using **Station_Master** as your template (it requires the same fields and data types).
2. Add a **named** primary key constraint to Master (using ID) after constructing the table.
3. Make the master's surname **compulsory**. Consider that you have already created the table.
4. Add all the existing data records from **Station_Master**. Elegance in your approach will be considered (e.g., efficient coding).
5. Insert the new station master records (only add the data that you know) as shown above.
6. Create a table called **Station_Extension** from **Station** ensuring all existing records are copied over to Station_Extension. StationID's are sequential and can continue from the last station in the station table.
7. Make the station name **unique and not-null**.

8. Add a **named** primary key constraint to Station_Extension (using ID).
9. Add a **named** foreign key constraint from Station_Extension to Master.
10. **Insert** new records for each of the 10 stations as shown above. If the data isn't provided do not add imaginary data to the table. However, do continue the sequence for Station_ID, e.g., Esplanade has Station_ID = 18.
11. Create a **view** called StationStaff that lists all station names and IDs from the **Station_Extension** table along with their matching station master and station master ID from the **Master** table, sorted by station name.

PART B: PYTHAGORAS' THEOREM & POINT-IN-RANGE (19 MARKS)

The transport department is seeking to implement a measure of "directness". This is a simple equation that takes the known distance and divides it by the "as the crow flies" distance, which is the shortest distance between two points. E.g., if the distance between North Freomantle and Mosman Park were known to be 3.5 km and the "as the crow flies" was also 3.5 km, then the route is as direct as can be ($3.5/3.5 = 1$). However, if the "as the crow flies" distance was 1.75, then the index would be 2 (rather indirect). To get things started, assume at this stage you only need the SQL to work with the input of any two stations:

Task 1 (5 marks): "As the Crow Flies"

What is the shortest distance between Karrakatta and Perth stations based on their station rail vertices "as the crow flies"? In other words, using Pythagoras' theorem.

You will need to get the X and Y's from the rail_vertex table that relate to the stations (in the stations table).

To help, here's a snippet of the rail_vertex table:

	vertex_id integer	x double precision	y double precision
1	1	381308	6453001
2	2	381971	6453866
3	3	382250	6454044
4	4	382207	6454462
5	5	381967	6455101
6	6	381991	6455471
7	7	382077	6456317

Followed by a snippet of the station table:

	station_id integer	station_name character varying (20)	vertex_id numeric (3)	max_railcars integer	interchange character (1)	stn_master_id character (7)
1	1	Fremantle	1	4	Y	32JT729
2	2	North Fremantle	6	4	N	64TZ630
3	3	Victoria Street	9	4	N	64TZ630
4	4	Mosman Park	12	4	N	64TZ630
5	5	Cottesloe	14	4	N	64TZ630
6	6	Grant Street	17	2	N	64TZ630
7	7	Swanbourne	19	4	N	17BD422
8	8	Claremont	23	4	Y	17BD422

Vertex_id is a relatable field in both tables. This will test your understanding of joins/subqueries, aliases, and mathematical functions in SQL.

POINT-IN-RANGE

Figure 1 illustrates a bounding box (yellow rectangle) around the suburb of North Fremantle (green polygon). Its extent is controlled by the maximum and minimum x and y coordinates of North Fremantle. Bounding boxes are used all the time in spatial queries – e.g., returning all the imagery within a rectangle or to clip your data to an area of interest. In our case, let's assume that users of the train station or town planners may want to know things like how many shops are within the bounding box around North Fremantle, for example.



Figure 1 - Bounding box around the suburb of North Fremantle.

The following code can be used to create a view that will return a bounding box for each suburb. I suggest you reverse engineer it to understand the machinations, and run it to get an idea of the results. You will need it for the next tasks.

```
CREATE VIEW V_SUBURBBOUNDS AS
SELECT S.Suburb Suburb, S.ssc_code SCC, MIN(V.x_coord) MinX,
MAX(v.x_coord)      MaxX, MIN(v.y_coord) MinY, MAX(v.y_coord)
MaxY
FROM SUBURB_ATTRIBUTE S, SUBURB_VERTEX V, SUBURB_ARC A
WHERE V.arc_id = A.arc_id AND (A.left_poly = S.suburb_id OR
A.right_poly = S.suburb_id)
GROUP BY S.Suburb, S.ssc_code
ORDER BY S.Suburb;
```

Task 2 (5 marks)

Design a single SELECT statement that will find the names of all stations that are within (or partially within) the bounding box of any suburb whose name starts with any of **North**, **South**, **West**, **East**, or **Central**, and display along with the suburb(s) they lie within.

Task 3 (5 marks)

Design a single SELECT statement that will find the names of all **stations** whose name starts with any of **North, South, West, East, or Central** and display along with the suburb bounding box(es) they lie within. **NB: this task is asking for STATIONS starting with North, South, West, East or Central, as opposed to task 1, which asked for SUBURBS starting with those).**

Task 4 (4 mark)

Combine the results from both statements to give you records where both station and suburb names start with any of **North, South, West, East or Central**? Prove it by supplying the necessary SQL. If you don't have any results for Task 1 and Task 2, you should still attempt the question for part marks.

End of Part B.

PART C: POINT-IN-POLYGON (15 MARKS)

In Part C we will look at the more precise query of finding points within an irregular shaped polygon, such as the actual suburb (Figure 2).



Figure 2 - The suburb and surrounds of North Fremantle illustrating the complex nature of suburb boundaries.

The test for point-in-polygon is too complex for standard, purely relational SQL.

An alternative is to make use of POSTGIS to extend postgresSQL with spatial data types and the spatial query capability.

Add the POSTGIS extension (if it's not already set up – if it is, it'll tell you and all is fine).

```
CREATE EXTENSION postgis;
```

Please use the create statement below to set up the Suburb table.

```
DROP TABLE IF EXISTS SUBURB;  
CREATE TABLE SUBURB  
(Suburb VARCHAR(18),  
SSC_code NUMERIC(4),  
Shape GEOMETRY('POLYGON ', 28350),  
CONSTRAINT suburbPK PRIMARY KEY (Suburb)  
);
```

Assume that there is only one record for each suburb name.

A feature point or feature site is a place of interest for train travellers. For example, features in these tables could include tourist attractions, sports venues, cultural centres, hospitals and educational institutions. Some sites and points are related. For instance, Royal Perth Showgrounds would be a feature site as it covers a large area but it may have several related feature points for each of its entrances.

Please use the following create table statements to set them up:

```
DROP TABLE IF EXISTS FEATURESITE;
CREATE TABLE FEATURESITE
(Name VARCHAR(40),
Short_name VARCHAR(20),
Feature_id NUMERIC(3),
Shape GEOMETRY('POLYGON', 28350),
CONSTRAINT fSitePK PRIMARY KEY (Feature_id)
);
```

```
DROP TABLE IF EXISTS FEATUREPOINT;
CREATE TABLE FEATUREPOINT
(Name VARCHAR(30),
Feature_point_id NUMERIC(3),
Feature_id NUMERIC(3),
Position GEOMETRY('POINT', 28350),
CONSTRAINT fPointPK PRIMARY KEY (Feature_point_id),
CONSTRAINT fPtSiteFK FOREIGN KEY (Feature_id)
REFERENCES FeatureSite (Feature_id)
);
```

Task 1 (4 marks)

Insert some (somewhat) artificial data into the tables.

Insert the following into the Suburb table:

Suburb	SSC	Array
Perth	5684	391209,6464720,390919,6463010,392833,6463280,392902,6464290,391209,6464720
West Perth	5915	391209,6464720,390601,6465380,388158,6462760,389259,6460250,389372,6461870, 390919,6463010,391209,6464720

Insert the following into the FeatureSite table:

Name	Short_name	Feature_Id	Array
Kings Park Botanical Gardens	Kings Park	01	390845,6464092,389756,6464429,388415,6463055,388300,6462792,388317,6461851,389210,6461975,389854,6462269,390951,6463227, 390845,6464092
Shenton Park	Shenton Park	02	387592,6463375,387910,6463691 #I'm a rectangle.
Perth Convention and Exhibition Centre	Convention Centre	03	391353,6463695,391753,6463864 #I'm a rectangle too.

Insert the following into the FeaturePoint table:

Name	FeaturePtId	ID	Array
Kings Park Main Garden	01	01	390543,6463279
DNA Tower	02	01	390032,6462929
Lotterywest Family Area	03	01	390300, 6462989
Mueller Playground	04	02	387599, 6463399

Task 2 (2 marks)

After you've done all of the inserts, create a spatial index of type GIST on each of the three tables.

Task 3 (3 marks)

Create a connection to your database in ArcMap/ArcPRO or QGIS – your choice! Bring all three layers into view. Save the project as your full name (e.g. Todd_Robinson). Provide a screenshot of ArcMap/ArcPRO/QGIS (showing as a minimum - your layers, the project name, and the table of contents window).

Task 4 (3 marks)

Write the spatial SQL statement to return the name of all feature points together with (where it exists) the short name of the point's associated feature site(s).

Task 5 (3 marks)

Write a spatial SQL statement to return the name of all feature points (not feature sites) that fall within the suburb of **West Perth**.

End of Part C.

To Hand In:

Hand in your work as a single report, using Word. Use a formal report format: include at least a cover page, table of contents and list of references in your report and use headings to separate out different sections.

Use formatting conventions for all SQL statements – see Lecture 4.

A Question/Answer format is sensible and acceptable for this assignment. E.g., you can use the Task as the heading, paste SQL as text (we might want to run it) as your response to the task followed by a screen shot of the output to prove it worked as expected.

End of Assignment